

Luna Backend Technical Documentation

Project Overview

Luna is the web implementation of the research study titled "Forecasting Intra-individual Changes of Affective States Taking into Account Inter-individual Differences Using Intensive Longitudinal Data from a University Student Dropout Study in Mathematics" conducted by Augustin Kelava, Pascal Kilian, Judith Glässer, Samuel Merk, and Holger Brandt at the University of Tübingen. The goal of this web application is to understand and forecast student dropout in German universities using state-of-the-art psychometric methods. Web app backend is a Django-based backend application designed to manage university-related operations, including student management, module handling, and survey systems.

Technical Requirements

Core Requirements

- Python 3.9
- Django 4.2.5
- PostgreSQL Database
- Docker & Docker Compose

Key Dependencies

- Django Rest Framework 3.14.0 (API Framework)
- django-cors-headers 4.2.0 (CORS handling)
- psycopg2-binary 2.9.7 (PostgreSQL adapter)
- django-cron (Scheduled tasks)
- drf_yasg (API Documentation)
- python-decouple (Environment configuration)

Project Structure

```
luna_backend/
├── luna/                # Main application directory
│   ├── api/            # API endpoints and views
│   ├── core/           # Core functionality and models
│   ├── modelling/       # Data modeling and analysis
│   └── luna/            # Project settings and configuration
├── documentation/       # Project documentation
├── requirements.txt      # Python dependencies
├── Dockerfile           # Container configuration
├── docker-compose.yml    # Docker services configuration
└── entrypoint.sh        # Docker entrypoint script
```

Development Setup

- Clone the repository
- Install dependencies: `pip install -r requirements.txt`
- Configure environment variables in `.env`
- Run migrations: `python manage.py migrate`
- Start development server: `python manage.py runserver`

Docker Deployment

The application can be deployed using Docker:

```
docker-compose up --build
```

Admin Portal

The application provides a Django admin interface for administrative tasks:

Access

- URL: `/admin`
- Login with superuser credentials
- Accessible only to users with admin privileges

Django Applications

The project is organized into several Django applications:

Core App

- Contains base models and functionality
- Handles user authentication and permissions
- Manages university and faculty data structures

API App

- Provides REST API endpoints
- Handles API authentication and permissions
- Contains API documentation (Swagger/OpenAPI)

Modelling App

The modelling app is responsible for processing and analyzing survey data, providing insights and predictions.

Key Components

1. Models

- `SurveyResults` : Stores survey analysis results
 - Links to Student and Module
 - Tracks evaluation timestamps
 - Stores various analysis outputs (raw, smoothed, covariance matrices)
- `FormResults` : Manages form analysis results
 - Connected to StudentForm
 - Stores array of numerical results
 - Provides helper methods for student and form information

2. Analysis Components

- `modelling.py` : Core analysis algorithms
- `model_settings.py` : Configuration for analysis parameters
- `form_loadings.py` : Form data processing utilities
- `utils.py` : Helper functions for data processing

API Endpoints

The modelling app exposes several REST endpoints. You can find the documentation in the `swagger.json` file or inside `views.` and `urls.py` folder in the modelling app.

Kalman Filter Implementation

The modelling app uses a Kalman Filter for state estimation and smoothing of survey data. The implementation is found in `modelling.py`.

1. Class Structure

```
KalmanFilter(F=None, B=None, H=None, Q=None, R=None, P=None, x0=None)
```

2. Required Parameters

- `F` : State transition matrix
- `H` : Observation matrix
- `B` : Control input matrix (optional)
- `Q` : Process noise covariance
- `R` : Measurement noise covariance
- `P` : Initial state covariance
- `x0` : Initial state

3. Key Methods

- `predict()` : Predicts the next state
- `update(z)` : Updates state based on measurement
- `forward(observations)` : Processes sequence of observations
- `smooth(predictions_state, predictions_cov)` : Applies RTS smoothing

Environment Variables

The following environment variables are required for the application:

Database Configuration

```
PGHOST=<database_host>      # PostgreSQL host (default: localhost)
PGNAME=<database_name>      # Database name
PGUSER=<database_user>      # Database user
PGPASSWORD=<database_password> # Database password
PGPORT=<database_port>      # Database port (default: 5432)
```

Django Configuration

```
DEBUG=<True/False>          # Debug mode (set False in production)
ALLOWED_HOSTS=<hosts>      # Comma-separated list of allowed hosts
```

Authentication System

The application uses a custom token-based authentication system:

- JWT (JSON Web Tokens) for API authentication
- Email-based user authentication
- Role-based access control (Student, Lecturer, Admin)
- Token expiration and refresh mechanism

Scheduled Tasks (Cronjobs)

The application uses django-cron for scheduled tasks. Cronjobs are defined in the `cronjobs.py` file and mostly designed to populate surveys for students for each module they are enrolled in according to specified time intervals.

Core Models

User Management

1. User

- Custom user implementation extending Django's `AbstractBaseUser`
- Email-based authentication
- User types: Student, Lecturer, Admin
- Fields:
 - email (unique identifier)
 - first_name
 - last_name
 - is_verified
 - user_type
 - university (Foreign Key)

2. StudentUser

- Extended user profile for students
- Fields:
 - Personal information (name, birth date)
 - Academic details (abitur_note)
 - Language preferences
 - Financial support status

Academic Structure

1. University

- Represents educational institutions
- Basic fields: name, creation/update timestamps

2. Faculty

- Represents university faculties
- Connected to University model
- Basic fields: name, timestamps

3. Module

- Represents academic courses/modules
- Fields:
 - name, code
 - university (Foreign Key)
 - owners (Lecturer/Admin users)
 - semester type (Winter/Summer)
 - date ranges
 - status (Active/Inactive)

Relationships

- Students can be enrolled in multiple modules (StudentModule)
- Modules belong to universities and are owned by lecturers
- Forms and surveys are linked to specific students and modules

Features

- User Management
 - Create/Edit/Delete users
 - Manage user roles and permissions
 - Reset passwords
 - View user activity
- Data Management
 - Manage Universities and Faculties
 - Handle Module configurations
 - Review and manage surveys
 - Monitor form submissions
- System Configuration
 - Configure system settings
 - Manage authentication settings
 - View and control scheduled tasks

Environment Variables

The following environment variables are required for the application:

Database Configuration

```
PGHOST=<database_host>          # PostgreSQL host (default: localhost)
PGNAME=<database_name>          # Database name
PGUSER=<database_user>          # Database user
PGPASSWORD=<database_password>  # Database password
PGPORT=<database_port>          # Database port (default: 5432)
```

Django Configuration

```
DEBUG=<True/False>              # Debug mode (set False in production)
DJANGO_SECRET_KEY=<secret_key>  # Django secret key
ALLOWED_HOSTS=<hosts>          # Comma-separated list of allowed hosts
```